

Page 1 - Cover Page

Student name	Almila Kocaagaoğlu
Student number	230417379
Lesson name and number	COM206

Page 2 - Introduction

Project Title:

Fluent Note App - A Modern PyQt5 Note Management System

Research Question / Problem Statement:

How can we efficiently manage, store, and view daily notes in a user-friendly, modern desktop environment without relying on complex backend databases?

Objective of the Project:

The project aims to design an in-memory, GUI-based notebook application that allows users to seamlessly add, read, and delete notes using an intuitive interface driven by the PyQt5 framework.

Expected Outcome:

A working Python program utilizing PyQt5 that demonstrates event-driven programming, UI state management (via QStackedWidget), and dynamic list manipulation, packaged in a modern and fluid user interface.

Page 3 - Methodology

Approach to Problem Solving:

- **Control Flow:** The application starts on a main dashboard (My Notes Screen). The user can view existing notes, delete them dynamically, or navigate to the Note Editor Screen to

create or read a note.

- **Components:**
 - *Input:* Text fields (QLineEdit, QTextEdit) for note titles and content, alongside interactive push buttons.
 - *Processing:* A Python list acts as the temporary RAM storage (session state). Functions handle adding dictionaries to this list, removing items by index (with confirmation dialogs), and dynamically loading them into the UI.
 - *Output:* A customized QListWidget displaying formatted note titles, timestamps, and interactive delete buttons directly within the list rows.
- **Data Structure:** Notes are structured as Python dictionaries (key-value pairs for "title", "content", and "date") and are manipulated within a standard Python List.

Example Control Flow (List-based Note Manager):

1. Start program
2. Display main dashboard (Add New Note, Read Note, Delete Note)
3. User selects option:
 - If Add Note: Navigate to Editor Screen, input data, save to list, return to dashboard.
 - If Delete Note (Trash Icon): Prompt confirmation, remove from list by index, refresh UI.
 - If Read Note: Load selected note data from the list into the Editor Screen.
 - If Exit: Terminate program (Memory is cleared).
4. Repeat until exit

Page 4 - Technical Implementation

Programming Language:

Python (PyQt5)

Code Example (Core Logic Implementation):

```
def save_note(self):
    title = self.title_input.text().strip()
    content = self.content_input.toPlainText().strip()

    if not title:
        QMessageBox.warning(self, "Validation Error", "A note must
have a title!")
        return

    current_time = datetime.now().strftime("%H:%M - %d/%m/%Y")
    new_note = {
        "title": title,
```

```
        "content": content,
        "date": current_time
    }

    self.main_window.notes_storage.append(new_note)
    self.main_window.my_notes_screen.load_notes_to_list()
    self.back_main_success()

def delete_note(self, index):
    reply = QMessageBox.question(self, 'Delete Note',
                                "Are you sure you want to delete"
                                "this note?",
                                QMessageBox.Yes | QMessageBox.No,
                                QMessageBox.No)

    if reply == QMessageBox.Yes:
        del self.main_window.notes_storage[index]
        self.load_notes_to_list()
```